

DrawTalking: Towards Building Interactive Worlds by Sketching and Speaking

Karl Toby Rosenberg
New York University
USA
ktr254@nyu.edu

Rubaiat Habib Kazi
Adobe Research
USA
rhabib@adobe.com

Li-Yi Wei
Adobe Research
USA
liyiwei@acm.org

Haijun Xia
University of California, San Diego
USA
haijunxia@ucsd.edu

Ken Perlin
New York University
USA
perlin@nyu.edu



DrawTalking combines sketching and talking-out-loud.



DrawTalking enables improvisational creative computation tasks via sketching and speaking.

Figure 1: Our approach *DrawTalking* mediates sketching and talking-out-loud through direct manipulation, enabling many use cases across improvisational creative tasks via sketching and speaking.

ABSTRACT

We introduce DrawTalking, a prototype system enabling an approach that empowers users to build interactive worlds by sketching and speaking. The approach emphasizes user control and flexibility, and gives programming-like capability without requiring code. An early open-ended study shows the mechanics resonate and are applicable to many creative-exploratory use cases, with the potential to inspire and inform research in future natural interfaces for creative exploration and authoring.

CCS CONCEPTS

• **Human-centered computing** → **Graphical user interfaces; Natural language interfaces; Interaction techniques; Interactive systems and tools;**

KEYWORDS

creativity, sketching, play, programmability, multimodal, human-AI collaboration, prototyping

ACM Reference Format:

Karl Toby Rosenberg, Rubaiat Habib Kazi, Li-Yi Wei, Haijun Xia, and Ken Perlin. 2024. DrawTalking: Towards Building Interactive Worlds by Sketching and Speaking. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*, May 11–16, 2024, Honolulu, HI, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3613905.3651089>

1 INTRODUCTION

Sketching while speaking aids innovation, thinking, and communication, with applications in animation, game design, education, engineering, rapid prototyping, and many other creative and spontaneous activities [10, 44]. The combination enables us to think about and share anything through make-believe – including things that do not or cannot exist. We achieve this by assigning *representations* (sketches) to *semantic concepts* (objects, behaviors, properties) [43]. For example we might suspend our disbelief so a square represents anything from a house, to a playing card, a map, dog, or person.

This work is an attempt to realize a style of spontaneous interaction that seamlessly integrates sketching and talking-out-loud to

build interactive, explorable worlds and simulations, thus greatly increasing our range of computational expression [45]. Prior works in interactive sketching [14, 30, 33, 40, 42], language/AI-mediated interfaces [3, 12, 18, 25, 38, 46–48], and visual programming-adjacent interfaces or games [6, 9, 17, 24, 29, 31, 41] has laid valuable groundwork. However, they often assume that users have predetermined goals or require pre-built content, limiting spontaneity and accessibility. Tools focus on generating a specific output rather than facilitating an ongoing creative process [4]. They might enforce given representations of objects (e.g. realistic or specific sketch recognition). They might feature complex UI, and in the case of programming-oriented tools, require explicit programming knowledge (whether via text, nodes, or blocks).

In our prototype system, *DrawTalking*, users speak while freehand-sketching to create, control, and iterate on interactive visual mechanisms, simulations, and animations. Via speech and direct manipulation, the user names their sketches to provide semantic information to the system, and narrates desired behaviors and rules to influence the interactive simulation, as if by explaining to others or telling a story. Users can build and control interactive worlds by simple combinations of sketching and speaking. *DrawTalking* requires no preparation or staging, and supports the user in making changes to content and behavior anytime. By design, we balance AI-automation with user-direction such that the user is the one who chooses how to represent content, when to do operations, and what logic to define.

In sum, we contribute *DrawTalking*: a novel sketching+speech interaction that balances user-agency with machine automation. We demonstrate *DrawTalking* by prototyping a multi-touch application for the iPad. An early qualitative study of *DrawTalking* reveals its use for emergent creativity and playful ideation.

2 RELATED WORK

Our research is based on a confluence of advances in multimodal, sketching, and programming interfaces.

2.1 Natural Language-Adjacent Interfaces

Systems such as SHRDLU [46] and Put That There [3] pioneered the vision of employing natural language to communicate with computers. Due to recent advances in speech recognition and natural language understanding, the popularity of this interaction modality has exploded, and has been used in a wide range of domains. For example, VoiceCut [16] and PixelTone [18] allow users to speak short phrases or sentences to perform desired operations in image editing applications, but these applications are heavily domain-specific. Tools like WordsEye [5], Scones [12], and CrossPower [47] enable scene generation or content editing via language, and interfaces such as Visual Captions [25], RealityTalk [22], and CrossTalk [48] use language structure to make content appear during talks or conversations. However, many of these interfaces tend to assume that the user knows what they want to create in advance — i.e. an end-product with an initial goal. They require an initial phase in which the user must define content and behavior up-front. This could limit open-ended exploration during the creative process, when the user does not necessarily have an end-goal in mind. Further, the majority of such interfaces use language input to generate or spawn content

without the user in the loop. An alternative is to enable greater interactive control and definition of the behavior of the content. Our approach emphasizes flexibility and user-control during the creative process. The user can define and iterate on content at any point. Our prototype specifically supports definition of behaviors within an interactive simulation. Within this prototype, we explore language input, combined with direct manipulation, as a way of empowering the user to program and control scenes interactively.

2.2 Dynamic Sketching Interfaces

HCI researchers have extensively explored sketching interfaces for dynamic and interactive visualizations ever since the first graphical user interface (GUI) SketchPad [40] and William Sutherland's thesis, the forerunner of the visual programming language [41]. Many works use direct manipulation and sketching techniques to help users craft interactive behaviors and toolsets for illustrated animation, UI, and visual-oriented programs. For example, works by Kazi et al. [14, 15], Landay et al. [17], Saquib et al. [33], and Jacobs et al. [13] focused on mixing illustration, programming, and prototyping. Programming by-demonstration is featured in works such as K-Sketch [7] and Rapido [21]. Scratch [31] is a well-known visual programming environment mixing game-like interactions with user-provided content and images for a playful experience. [39] supports the user in forming connections between texts and concepts to learn via active diagramming. Our interface is framed in a complementary way around correspondences between sketches and language elements, but we use sketch-language mapping as a control mechanism, enabling the user to create interactive simulations and behaviors for open-ended exploration.

Prior work also explored supporting development of pre-programmed simulations and domain specific behaviors to craft interactive diagrams. In Chalktalk, for example, the system uses sketch recognition to map a user's hand drawn sketches into corresponding dynamic, pre-programmed behaviors/visualizations [30]. More domain-specific tools like MathPad2 [19], Eddie [34], PhysInk [35] and SketchStory [20] use hand-drawn sketches and direct manipulation interactions to create interactive simulations in physics, math, and data visualization.

2.3 Programming-Like Interfaces

The customizability and flexibility of such a more general interface implies a need for programmability. Examples of these include real-time world simulation systems and programmable environments such as the SmallTalk programming language [11], Scratch [26, 31] (and similar AI voice-enabled explorations like StoryCoder [8]), Improv [29], ChalkTalk [30], and creative world-building games like the Little Big Planet series [23, 24, 32] and Dreams [9]. These encourage interactive building of scenes, games and stories. They combine with elements of interactive visual programming and drawing/sculpting with many types of content (2D, 2.5D, 3D, images). But all use explicit interfaces for programming or programming-like functionality (nodes, wires, text). We depart from explicit UI for sketching+programming-like capability, and largely replace many programming-like functionality with the use of language. Our direction explores the use of verbal, descriptive story narration together

with other input modalities (i.e., touch and pen input) to create animated and interactive graphics through sketching.

3 DESIGN GOALS

In brief, we envision an interface for creative exploration with interactive capability that (a) does not impose many assumptions about the user’s intent or content, (b) focuses on the process, not just on the artifact, and (c) does not require programming knowledge. Above all, the user should have control. To that end, we wanted an interface that:

- *has minimal system assumptions* where the user controls the representation and behavior for sketches
- *is flexible, mutable, and fluid* in that it supports improvisation, quick changes, and rapid iteration of ideas, where operations are easily accessible and doable in any order
- *supports multiple opportunities for error-recovery* by telegraphing what the system’s understanding is
- *supports many language primitives that can be parameterized and re-combined* without the need for a coding interface

4 DRAWTALKING

In DrawTalking, users freehand-sketch on a digital canvas and simultaneously speak to narrate. Speaking serves a dual-purpose: the user can explain concepts and tell stories, and at the same time refer to and label the objects in their world with semantics (nouns, adjectives, adverbs) determining their names and behaviors. This labeling tells the system what the objects are, irrespective of their visual representation, and can be changed at any time – inspired by our ability to ‘make-believe’. Once labeled, sketches can be controlled via narration or typing. Rules governing the logic in the simulation can be created to automate behavior, and touch controls also allow the user to interact directly with the simulated world. This results in a freeform sandbox for animation and programming-like behavior that mixes direct user-control with machine automation.

4.1 Sketching & Language Interface Elements

DrawTalking’s interface has a standard pen+multitouch tablet app design with a manipulable infinite canvas. Sketches are independently-movable freehand-drawings, text, or numbers created by the user.

DrawTalking receives a subset of English as input. We built-in primitives for verbs, adjectives, and adverbs for playful interactions, e.g. procedural transformations and movement, edit operations, collisions, user input. These are composable into more complex behavior. These primitives are intended to be a small sample of possible functionality that demonstrate our working concept. (They are partly inspired by existing software and design spaces [9, 31, 36].) We do not account for all of natural language or claim that our implementation is the only solution. Rather, in the scope of our research our main contribution is the interactive way of controlling elements, independent of the fidelity and comprehensiveness of the supported behaviors. Authoring completely new behaviors sans programming is out-of-scope, but is a complementary future direction. That said, we provide several ways to build on-existing behaviors. We note that the result was just one possible instantiation of our

concept. It was most convenient to prototype an all-in-one interface we could fully customize. Other interface implementations could use a similar control mechanism, while integrating more advanced visuals and ways to author new behaviors to control.

The user labels their sketches to make them controllable: nouns (names) for unique identification and adjectives and adverbs (properties) for modulating sketch behavior. For flexibility, we offer two direct ways to label (or unlabel):

- (1) tap 1+ sketches and speak with deixis [37] (e.g. *"this/that is a <blank>"*, *"this/those are <blank>s"*).
- (2) link sketches with words via touch+pen (enabling freer narration).

The labels display adjacent to sketches, similar to whiteboard annotation, and are quickly-removable by touch.

Consistent with all design goals, the interface (Figure 2) exposes a *transcript* view and semantics diagram (Figure 2b) to make received input transparent, quickly editable and accessible, and error-robust. The former lets the user visualize speech input, link sketches, and stage/confirm commands. Speech recognition is continuous for interactive use. The design places user-control first, so the user decides when speech is ready as a command. The user taps ‘language action’ to stage a command. Then the diagram visualizes the machine’s understanding of the input and provides a way to reassign objects within the command, regardless of location. The user confirms with the same button to execute the command, or taps the discard button to cancel.

4.2 Language Commands

DrawTalking interprets the structure of language input into commands built from primitives.

Verbs are actions performable by sketches and the system, either built-in or user-defined in terms of other verbs. Examples of implemented verb primitives include:

- animations (e.g. *move, follow, rotate, jump, flee*),
- state changes (e.g. *create, transform*),
- events (e.g. *collides with, press*),
- inequalities (e.g. *equal, exceed*).

Verb behavior changes based on other parts of the sentence:

Conjunctions run simultaneously (e.g. *"The dog jumps **and** the cats jump"*).

Sequences run in-order (e.g. *"The dog jumps **and then** the cats jump"*).

Stop commands cancel an ongoing operation (e.g. *"The square stops moving"*).

Prepositions (like *on, to, under*) cause verbs to exhibit different behavior, e.g. *"The dog jumps **on/under** the bed"* impacts the dog’s final position.

Timers specify the duration of a verb, e.g. *"the square moves up for 11.18 seconds and then jumps."*

Loops repeat an action, either forever (e.g. *"endlessly the dog jumps"*), or finitely (e.g. *"10 times the dog jumps excitedly"*).

Special verbs include *"become"*, which modifies the sketch’s labels, and *"transform into"*, which also instantly replaces a sketch’s visual representation with another’s. This can be used to support state

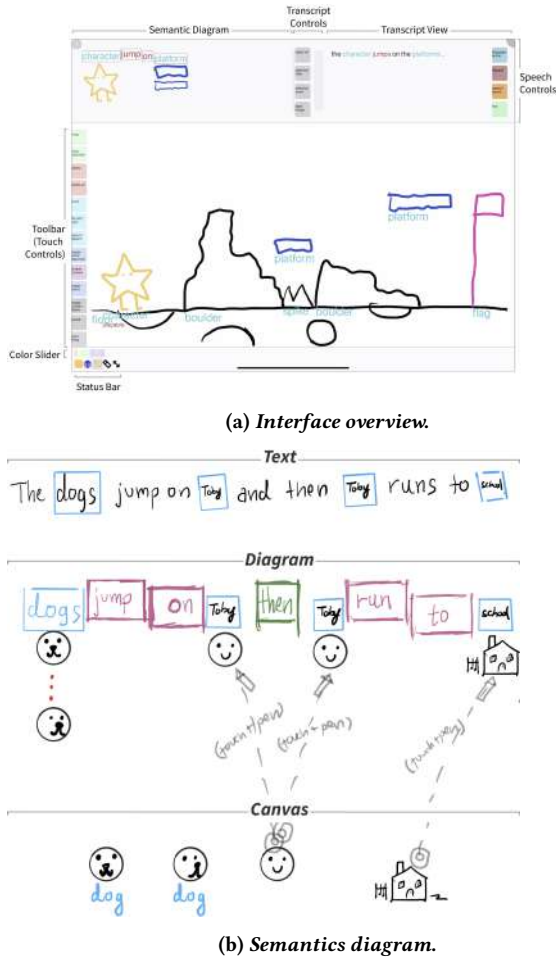


Figure 2: Left (Figure 2a) is an interface screenshot (taken from P4 in section 5). In it, the utterance “The character jumps on the platforms” selects the sketch labeled ‘character’ and all sketches labeled ‘platform.’ Tapping the ‘language action’ button (top-right) will confirm the command and cause the character to jump on all platforms; the toolbar enables edit operations, e.g. copy, delete, save (to save and spawn sketches). To the right (Figure 2b) is an illustration of the semantics diagram. It shows what the system understood to help the user validate and optionally modify their input. Each diagram noun has a vertical list of ‘proxy’ icons referring to machine-selected objects. Erasing the proxy erases the original wherever it is. Pen+touch between diagram nouns and objects in the scene allows for relinking to modify or correct the machine’s selections. As shown, 2 dogs exist, but no ‘Toby’ or ‘school.’ The user can connect unlabeled sketches to complete the command. If a verb is unknown, the user can pick from a presented list of similar verbs or cancel.

changes, e.g. “the sun transforms into a moon” or “the frog transforms into a prince”.

Nouns, pronouns, and deixis refer to object labels and are used to pick *specific* objects or specify *types* of objects. Using deixis while

selecting an object will select the object immediately; pronouns can refer-back to objects, enabling more natural sentences as commands. For choosing **specific** sketches in a command:

- choose 1 or more specific object instances: “The” + noun
- choose all of a label: “all” + nouns
- select a specific number of objects: <number> + noun(s)
- select a random object: “a” + noun

There are a few special cases. The pronoun “I” is reserved; it allows the user to take-part in the narrative of a command (e.g. “I destroy the wall”). “Thing” is also a reserved noun that can refer to any object regardless of label. Plural nouns with no modifiers (e.g. as in “blades rotate”) refer to labels used to define the interactions between objects with that label. Specifically, this is used to construct rules, as described below and in Figure 4.

Adjectives and adverbs are usable as labels that define properties on objects. They’re interpreted by verbs continuously to modulate effects (e.g. magnitude, speed, size, distance, height). “fast”, “slow”, “excited”, for example, impact jump height and/or movement speed. Adjectives also disambiguate like-noun labeled-objects (e.g. “first house” vs. “second house”).

Adverbs heighten adjective effects multiplicatively – e.g. “very” and “slightly” – and are chainable – e.g. “very, very.” Special adjectives offer system-control: e.g. “static” fixes sketches to the screen like a GUI element, useful for buttons, d-pads, score displays, etc.; “visible”/“invisible” toggle sketch visibility.

Rules are conditionals that run commands in the future when objects with certain labels satisfy the condition. This enables the user to specify automated commands for the future when, as, or after an event has completed, without needing to know what they want in advance, e.g., “When arrows collide with balloons arrows destroy balloons.” Rules allow definition of new verbs in terms of existing ones e.g. if eat is not defined: “When things eat objects things move to objects and then things destroy objects.”

4.3 Putting It All Together by Example

We demonstrate DrawTalking with illustrated examples showing multiple procedural and programming-like capabilities.

4.3.1 Pond scene. A simple example demonstrating layered and random behavior was given earlier in Figure 1. Here a user sketches a frog, lily pads, water, and a butterfly. For random hopping, they say: “Forever the frog hops to a lily,” and compose it with commands for butterfly to chase the frog, but adjust the butterfly’s speed with “this butterfly is slow.” The user can pause the simulation to edit such moving objects easily.

4.3.2 Dog and boy’s infinite game of fetch. This example of looped sequences highlights a key design decision: being able to iterate on a scene and play with it anytime restarting or knowing the desired content in advance. The user begins by sketching a boy, dog, water, and ball (Figure 3). They can then interactively move the objects to influence the simulation. For additional effect, the water could rise upon collision with the ball, with a command like: “When water collides with balls water moves up for 0.2 seconds and then water moves down for 0.2 seconds.” The user can also move the objects to affect their behavior.

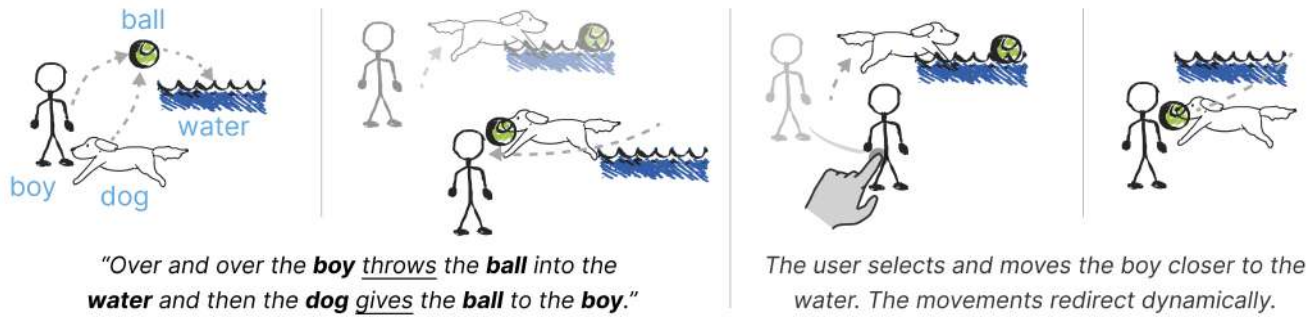


Figure 3: Dog and boy’s infinite game of fetch. Left: Labeled sketches are commanded. Right: Interactive user-participation.

4.3.3 Windmill simulation. This example demonstrates rules, custom object saving, and spawning (Figure 4). The user begins by drawing a windmill base and attaches a blade sketch. Then, they draw their own wind spiral and save it. They then command the blades to rotate on collision with the wind and immediately see the result by moving their wind object over the blades. They would notice the blades don’t stop. They may quickly fix that by adding an “after” stopping condition as shown. Now, they create a switch sketch and a wall, and finally configure the switch to spawn rightward-moving wind that causes the windmill blades to spin on-contact.

4.3.4 Creating the game “Pong” and turning it into “Breakout”. We now describe a more complex example of iteration demonstrating instant playtesting functionality reuse: quickly turning Pong [1] into Breakout [2] (Figure 5). For clarity, some detail is omitted. Create a ball and walls, then paddles, goals, and points for each player. For points, say “I want the number 0” and touch+pen the number to spawn a number object. Label with “This is the first/second score, goal”. Make points screen-space UI with “This thing is static” and set-up rules for collisions between the ball and goals: “When balls collide with first/second goals second/first scores increase”. Last, add collision logic: “when balls collide with walls walls reflect balls”, “when balls collide with paddles paddles reflect balls.” Now we have a playable touch-based version of Pong.

We transform this into Breakout by rotating the canvas, deleting the second-player-related sketches, and simply adding a blocks mechanic. To speed-up our drawing process, we can sketch a temporary rectangular ‘region’ and say “I pack the region with blocks” to fill the region with a sketch e.g. a custom ‘block’ sketch. To make the blocks destructible and increase the point count, create a rule “When balls collide with blocks balls destroy blocks and then the score increases.” We are done. In a few steps we have been able to build-on the same scene to try multiple variants of the game.

4.4 Implementation in-Brief

We implemented DrawTalking natively on the iPad Pro M2. Speech recognition is on-device. A local NodeJS+Python serverside for NLP runs on a macbook pro. Text is continuously sent to the server where 1) a dependency-tree is created via Spacy 3.1.4 [27], 2) sent to the client and compiled into a key-value format for semantic roles (e.g. AGENT, OBJECT), 3) either interpreted in real-time by

our custom engine to drive interactive simulations or used to find objects by deixis.

5 OPEN-ENDED USER STUDY

We conducted an exploratory discussion-focused study to gauge understandability of interaction, discover use cases and directions, and learn users’ perceptions from their experience. We invited 9 participants (students, professors, artists, game design). Each study lasted 1 hour and 15 minutes, and each participant was compensated by 30 USD.

For each session, the researcher sat alongside the participant and taught the drawing features. Next, the participant was told to draw 5 objects of their choice, then taught both labeling methods. Then, the session was improvised using the objects to explore all language features in rough increasing complexity. The researcher could help or suggest ideas, but participants mainly guided their own exploration using provided a language features list. They were allowed to think-aloud and comment. After, the participant was asked to reflect on the experience.

We chose this approach because we were interested in qualitative, early stage feedback on the *concept* and *interactions* behind DrawTalking, rather than feedback on our particular implementation. In effect, our study did not capture metrics comparing our specific DrawTalking system against other baseline systems. This is due to the collaboration between the researcher and participants in exploring the interface during the study. Furthermore, there is no known baseline to compare against, as any participant with experience with one tool or another might have had expectations that mismatched with our experimental interactions. Quantitative-based performance comparisons would also make it too easy to fall into the “usability trap” as described in Olsen’s work [28], as DrawTalking is a prototype, not a feature complete production tool, or a perfectly-designed interface. Furthermore, our study does not evaluate DrawTalking’s effectiveness for a specific task, as we were looking for user feedback on potential use cases. In our exploratory sessions, we let participants draw their own conclusions as to the usefulness, use cases, and potential for the interactions. We note that evaluating DrawTalking’s interactions might require more longitudinal-style studies in specific domains e.g. classroom environment for specific types of lessons on physical phenomena. Although limited in scope, the exploratory interface matched well for what we wanted to learn.

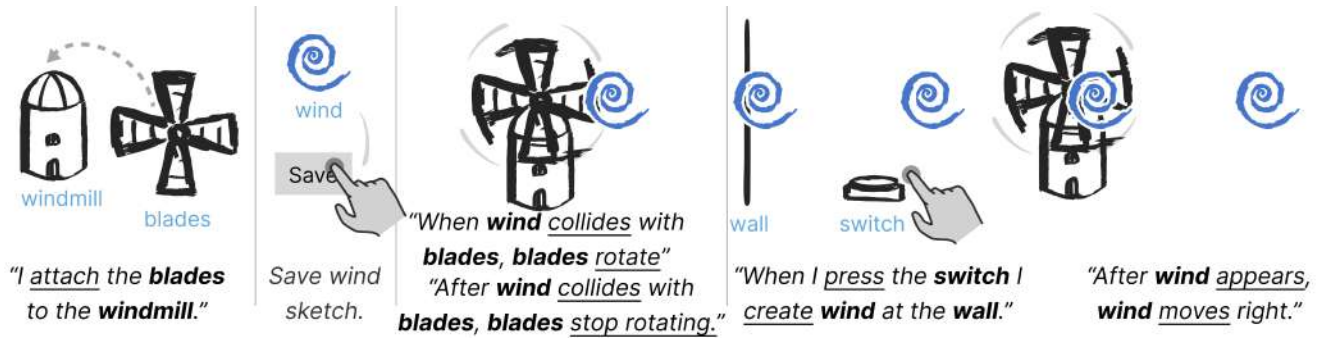


Figure 4: Windmill Simulation. Flexible process for constructing an interactive windmill built from user-defined rules, sketches, and triggers. The user can do this in *any* order and immediately try results at each step. This works on *any* sketch labeled "blade."

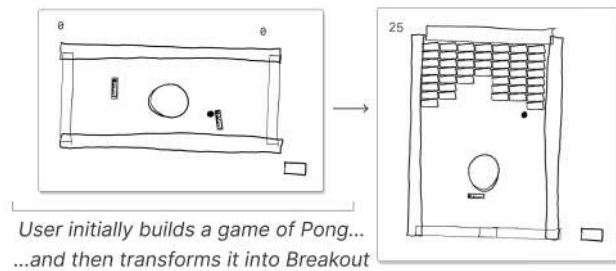


Figure 5: Pong into Breakout. *Left*: per-player paddles and points; *Right*: second player objects removed, destructible bricks added.

5.1 Results

All participants understood and learned the mechanics, and produced thoughtful discussion. They tried their own ideas for how to make things or test their own understanding out of curiosity. *All became comfortable.* They identified use cases include educational applications, rapid videogame / rapid prototyping, paper prototyping, user interface design, presentation, language learning, and visual-oriented programming.

5.2 Qualitative Findings

Below, we explore the qualitative experiences of participants using DrawTalking.

Labeling. We observed labeling by speech (deixis) was unanimously preferred over linking. Users called it intuitive, direct, and similar to how people talk. Linking could be a useful fallback when freer grammar is preferred, e.g. when giving talks and presentations.

Semantics diagram. People understood it and found it useful to introspect commands: P7: "[It's] showing what it's going to do. It's taking the verbs and cutting all the other stuff out." P2: "You might say something wrong and it interprets it in a different way and you can correct it." P8: "I immediately understand what happens when I read it."

Interactivity/Physicality. Users reported a strength of DrawTalking was a sense of physicality: P5: "I can execute different rules of things that are happening. scene is playing out here AND physical thing is happening." P1: "It's like language and Legos put-together." P2: "When we make games there's something called a paper prototype where we make a bunch of pieces of the players and the objects. We just move it around by hand to kind of simulate what it would be. [DrawTalking] is kind of like that but on steroids a bit. So it's very nice to be able to kind of have those tools to help you with it without needing to manually make it."

As a generic toolset, Re-applicability. Users saw potential for DrawTalking as generic functionality for integration with other applications in a creative pipeline, e.g. as a playful ideation phase from which to import/export assets or runnable code; attaching speech control to production tools to support faster workflows. P1: "You could sit here and have a conversation and build up just using language your interactions and then you send that out for code." P3: "here, it just takes one second for me to 'say it' – [this could be a] built-in function for any tool/interactive AI". P8 "just incorporate the language and control interface here. We can easily create animations with the fancy stuff they have." This suggests users perceive DrawTalking as a useful interaction, complementary to others in many contexts.

As a programming-like environment. Several participants immediately drew comparisons between primitives in DrawTalking to constructs like variables and loops. P7, an experienced digital artist, but a non-programmer, appreciated DrawTalking as an accessible tool that could reduce frustration encountered with existing programming tools: "Me instructing a game engine – someone like me who's not a programmer and who is intimidated by doing C# or ... using [a] visual scripting language like blueprints – this is a really clean interface that I think can achieve you can get 90- or 80% the way there. It just makes the user experience cleaner than having to use all these knobs and buttons or things like that or using scripting language or having to actually write code".

6 DISCUSSION, FUTURE WORK

In sum, users described DrawTalking as fluid and flexible; a natural language way of achieving programming-like functionality; a

rapid prototyping environment; an independent general interaction technique; capable of integration with other applications; physical, tangible, spatial; accessible to kids.

We believe DrawTalking works, then, because it successfully captures some of the playful and creative attributes of programming, spatial manipulation and sketching and language, owing to our initial goals and to our designs.

We emphasize that our primary contribution is the interaction mechanism enabling the user to speak and control elements interactively. The secondary contribution, the specific DrawTalking system we have presented, is an instantiation of that concept as a prototype. The prototype has limitations linked to the underlying implementation details. The current prototype requires extensions to the language compiler to process more variations of English. The interactive primitives cover many features in existing game engines; we and participants were able to use them creatively and in combinatoric ways, but they are visually simplistic. Adding new animations and functions requires programming knowledge to extend the system using an underlying high-level scripting API or lower-level code (but such new primitives can do anything independent of the controls, as they are built from raw programming language code). The control mechanism does not depend on what the primitives are, only on whether the mappings between language and primitives exist. A more advanced implementation could account for many more precise levels of control.

There are many possible directions: Improving the system design and vocabulary, and building on the strengths of the visual mapping; integrating with other applications to facilitate longer-term studies into creative workflows using our approach; multiuser collaboration. We'd also like to see how DrawTalking concepts could extend to augmented reality for interaction with tangible objects. Participants suggested that language models could offer supplemental functionality and support even more natural speech: generative language models as of writing are too slow for interactive time, but in the future might be used to convert fully natural language into structured subsets easy for systems like ours to parse, or to generate commands based on implicit context to reduce user-specification.

To address the limitations of the current implementation of DrawTalking, we could explore ways to support:

language input that is more natural. This could potentially be addressed with a more robust and feature-complete backend: e.g. improve the language processing implementation and/or integrate with faster generative language models for translation to simpler instructions.

spontaneous authoring of completely new primitives without (or with little) programming knowledge. This was out-of-scope for this project, and much research has tried achieving varying levels of programmability without requiring code. Nevertheless, exploring interactions for richer behavior-authoring interactions compatible our approach would be an excellent direction. We can imagine a number of possibilities: exploring extensions to the existing semantics diagram interface; using DrawTalking's controls to drive other applications with their own primitives; combining with the capabilities of generative models; crowd-sourcing programmed-functionality through shared libraries of domain-specific functionality; introducing multiple layers of programming capability in the same interface for different audiences (e.g. artists, programmers),

trading-off simplicity for granular control as is common in many applications.

We hope that any solution will consider how best to retain the feeling of user-control and user-participation. This project, although limited in terms of implementation, has been an attempt to probe potential first steps.

7 CONCLUSION

We have introduced and implemented our approach, DrawTalking, to building interactive worlds by sketching and speaking, and have shown its potential. Our interface was partly inspired by our natural interplay between sketching, language, and our ability to communicate via make-believe. There are many exciting directions and we would be excited to see future research build on our approach or uncover other human-centered approaches to extend natural human abilities. We consider this project just one possible step forwards that we hope fosters fruitful discussion and research.

ACKNOWLEDGMENTS

We would like to thank the anonymous reviewers for their constructive feedback. We also give special thanks to Devamardeep Hayatpur for additional feedback on writing and visuals for the paper.

REFERENCES

- [1] 2022. 50 Years of Fun With Pong. <https://computerhistory.org/blog/50-years-of-fun-with-pong/> Section: Curatorial Insights.
- [2] AtariAdmin. 2022. New Insight into Breakout's Origins. <https://recharged.atari.com/the-origins-of-breakout/>
- [3] Richard A. Bolt. 1980. Put-that-there: Voice and gesture at the graphics interface. *ACM SIGGRAPH Computer Graphics* 14, 3 (July 1980), 262–270. <https://doi.org/10.1145/965105.807503>
- [4] K. Compton and M. Mateas. 2015. Casual Creators. <https://www.semanticscholar.org/paper/Casual-Creators-Compton-Mateas/f9add8f5126faab72c9cc591b5fdc7e712936b56>
- [5] Bob Coyne and Richard Sproat. 2001. WordsEye: an automatic text-to-scene conversion system. In *Proceedings of the 28th annual conference on Computer graphics and interactive techniques (SIGGRAPH '01)*. Association for Computing Machinery, New York, NY, USA, 487–496. <https://doi.org/10.1145/383259.383316>
- [6] cycling74. 2018. Max. <https://cycling74/products/max>
- [7] Richard C. Davis, Brien Colwell, and James A. Landay. 2008. K-sketch: a 'kinetic' sketch pad for novice animators. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08)*. Association for Computing Machinery, New York, NY, USA, 413–422. <https://doi.org/10.1145/1357054.1357122>
- [8] Griffin Dietz, Nadin Tamer, Carina Ly, Jimmy K Le, and James A. Landay. 2023. Visual StoryCoder: A Multimodal Programming Environment for Children's Creation of Stories. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*. Association for Computing Machinery, New York, NY, USA, 1–16. <https://doi.org/10.1145/3544548.3580981>
- [9] Dreams 2020. Dreams. [Playstation 4]. <https://www.mediamolecule.com/games/dreams> <https://indreams.me>
- [10] Judith E. Fan, Wilma A. Bainbridge, Rebecca Chamberlain, and Jeffrey D. Wammes. 2023. Drawing as a versatile cognitive tool. *Nature Reviews Psychology* 2, 9 (Sept. 2023), 556–568. <https://doi.org/10.1038/s44159-023-00212-w> Number: 9 Publisher: Nature Publishing Group.
- [11] Adele Goldberg and David Robson. 1983. *Smalltalk-80: the language and its implementation*. Addison-Wesley Longman Publishing Co., Inc., USA.
- [12] Forrest Huang, Eldon Schoop, David Ha, and John Canny. 2020. Scones: towards conversational authoring of sketches. In *Proceedings of the 25th International Conference on Intelligent User Interfaces (IUI '20)*. Association for Computing Machinery, New York, NY, USA, 313–323. <https://doi.org/10.1145/3377325.3377485>
- [13] Jennifer Jacobs, Joel R. Brandt, Radomir Meëh, and Mitchel Resnick. 2018. Dynamic Brushes: Extending Manual Drawing Practices with Artist-Centric Programming Tools. In *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems (CHI EA '18)*. Association for Computing Machinery, New York, NY, USA, 1–4. <https://doi.org/10.1145/3170427.3186492>
- [14] Rubaiat Habib Kazi, Fanny Chevalier, Tovi Grossman, and George Fitzmaurice. 2014. Kitty: sketching dynamic and interactive illustrations. In *Proceedings*

- of the 27th annual ACM symposium on User interface software and technology (UIST '14). Association for Computing Machinery, New York, NY, USA, 395–405. <https://doi.org/10.1145/2642918.2647375>
- [15] Rubaiat Habib Kazi, Fanny Chevalier, Tovi Grossman, Shengdong Zhao, and George Fitzmaurice. 2014. Draco: bringing life to illustrations with kinetic textures. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '14)*. Association for Computing Machinery, New York, NY, USA, 351–360. <https://doi.org/10.1145/2556288.2556987>
 - [16] Yea-Seul Kim, Mira Dontcheva, Eytan Adar, and Jessica Hullman. 2019. Vocal Shortcuts for Creative Experts. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*. Association for Computing Machinery, New York, NY, USA, 1–14. <https://doi.org/10.1145/3290605.3300562>
 - [17] James A. Landay. 1996. SILK: sketching interfaces like crazy. In *Conference Companion on Human Factors in Computing Systems (CHI '96)*. Association for Computing Machinery, New York, NY, USA, 398–399. <https://doi.org/10.1145/257089.257396>
 - [18] Gierad P. Laput, Mira Dontcheva, Gregg Wilensky, Walter Chang, Aseem Agarwala, Jason Linder, and Eytan Adar. 2013. PixelTone: a multimodal interface for image editing. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '13)*. Association for Computing Machinery, New York, NY, USA, 2185–2194. <https://doi.org/10.1145/2470654.2481301>
 - [19] Joseph J. LaViola and Robert C. Zeleznik. 2004. MathPad2: a system for the creation and exploration of mathematical sketches. In *ACM SIGGRAPH 2004 Papers (SIGGRAPH '04)*. Association for Computing Machinery, New York, NY, USA, 432–440. <https://doi.org/10.1145/1186562.1015741>
 - [20] Bongshin Lee, Rubaiat Habib Kazi, and Greg Smith. 2013. SketchStory: Telling More Engaging Stories with Data through Freeform Sketching. *IEEE Transactions on Visualization and Computer Graphics* 19, 12 (Dec. 2013), 2416–2425. <https://doi.org/10.1109/TVCG.2013.191> Conference Name: IEEE Transactions on Visualization and Computer Graphics.
 - [21] Germán Leiva, Jens Emil Grønbaek, Clemens Nylandsted Klokmoose, Cuong Nguyen, Rubaiat Habib Kazi, and Paul Asente. 2021. Rapido: Prototyping Interactive AR Experiences through Programming by Demonstration. In *The 34th Annual ACM Symposium on User Interface Software and Technology (UIST '21)*. Association for Computing Machinery, New York, NY, USA, 626–637. <https://doi.org/10.1145/3472749.3474774>
 - [22] Jian Liao, Adnan Karim, Shivesh Singh Jadon, Rubaiat Habib Kazi, and Ryo Suzuki. 2022. RealityTalk: Real-Time Speech-Driven Augmented Presentation for AR Live Storytelling. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology (UIST '22)*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3526113.3545702>
 - [23] Little Big Planet 2008. Little Big Planet. [Playstation 3]. <https://www.mediamolecule.com/games/littlebigplanet>
 - [24] Little Big Planet 2 2011. Little Big Planet 2. [Playstation 3]. <https://www.mediamolecule.com/games/littlebigplanet2>
 - [25] Xingyu Bruce Liu, Vladimir Kirilyuk, Xiuxiu Yuan, Peggy Chi, Xiang 'Anthony' Chen, Alex Olwal, and Ruofei Du. 2023. Visual Captions: Augmenting Verbal Communication with On-the-fly Visuals. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI)*.
 - [26] John Maloney, Mitchel Resnick, Natalie Rusk, Brian Silverman, and Evelyn Eastmond. 2010. The Scratch Programming Language and Environment. *ACM Transactions on Computing Education* 10, 4 (Nov. 2010), 16:1–16:15. <https://doi.org/10.1145/1868358.1868363>
 - [27] Ines Montani, Matthew Honnibal, Matthew Honnibal, Sofie Van Landeghem, Adriane Boyd, Henning Peters, Paul O'Leary McCann, jim geovodi, Jim O'Regan, Maxim Samsonov, György Orosz, Daniël de Kok, Duygu Altinok, Søren Lind Kristiansen, Madeesh Kannan, Raphaël Bournhonesque, Lj Miranda, Peter Baumgartner, Edward, Explosion Bot, Richard Hudson, Raphael Mitsch, Roman, Leander Fiedler, Ryn Daniels, Wannaphong Phatthiyaphaibun, Grégory Howard, Yohei Tamura, and Sam Bozek. 2023. explosion/spaCy: v3.5.0: New CLI commands, language updates, bug fixes and much more. <https://doi.org/10.5281/zenodo.7553910>
 - [28] Dan R. Olsen. 2007. Evaluating user interface systems research. In *Proceedings of the 20th annual ACM symposium on User interface software and technology (UIST '07)*. Association for Computing Machinery, New York, NY, USA, 251–258. <https://doi.org/10.1145/1294211.1294256>
 - [29] Ken Perlin and Athomas Goldberg. 1996. Improv: a system for scripting interactive actors in virtual worlds. In *Proceedings of the 23rd annual conference on Computer graphics and interactive techniques (SIGGRAPH '96)*. Association for Computing Machinery, New York, NY, USA, 205–216. <https://doi.org/10.1145/237170.237258>
 - [30] Ken Perlin, Zhenyi He, and Karl Rosenberg. 2018. Chalktalk : A Visualization and Communication Language – As a Tool in the Domain of Computer Science Education. <https://doi.org/10.48550/arXiv.1809.07166> arXiv:1809.07166 [cs].
 - [31] Mitchel Resnick, John Maloney, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, and Yasmin Kafai. 2009. Scratch: programming for all. *Commun. ACM* 52, 11 (Nov. 2009), 60–67. <https://doi.org/10.1145/1592761.1592779>
 - [32] J. Ross, Oliver Holmes, and Bill Tomlinson. 2012. Playing with Genre: User-Generated Game Design in LittleBigPlanet 2. <https://www.semanticscholar.org/paper/Playing-with-Genre%3A-User-Generated-Game-Design-in-2-Ross-Holmes/75f36eb8585d9d7039a98c750b0085cc973eb689>
 - [33] Nazmus Saquib, Rubaiat Habib Kazi, Li-yi Wei, Gloria Mark, and Deb Roy. 2021. Constructing Embodied Algebra by Sketching. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI '21)*. Association for Computing Machinery, New York, NY, USA, 1–16. <https://doi.org/10.1145/3411764.3445460>
 - [34] John Sarracino, Odaris Barrios-Arciga, Jasmine Zhu, Noah Marcus, Sorin Lerner, and Ben Wiedermann. 2017. User-Guided Synthesis of Interactive Diagrams. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems (CHI '17)*. Association for Computing Machinery, New York, NY, USA, 195–207. <https://doi.org/10.1145/3025453.3025467>
 - [35] Jeremy Scott and Randall Davis. 2013. Physink: sketching physical behavior. In *Adjunct Proceedings of the 26th Annual ACM Symposium on User Interface Software and Technology (UIST '13 Adjunct)*. Association for Computing Machinery, New York, NY, USA, 9–10. <https://doi.org/10.1145/2508468.2514930>
 - [36] Yang Shi, Zhaorui Li, Lingfei Xu, and Nan Cao. 2021. Understanding the Design Space for Animated Narratives Applied to Illustrations. In *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems (CHI EA '21)*. Association for Computing Machinery, New York, NY, USA, 1–6. <https://doi.org/10.1145/3411763.3451840>
 - [37] Andreea Stapleton. 2017. Deixis in Modern Linguistics. *Essex Student Journal* 9, 1 (Jan. 2017). <https://doi.org/10.5526/esj23> Number: 1 Publisher: University of Essex Library Services.
 - [38] Hariharan Subramonyam, Wilmot Li, Eytan Adar, and Mira Dontcheva. 2018. TakeToons: Script-driven Performance Animation. In *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology (UIST '18)*. Association for Computing Machinery, New York, NY, USA, 663–674. <https://doi.org/10.1145/3242587.3242618>
 - [39] Hariharan Subramonyam, Colleen Seifert, Priti Shah, and Eytan Adar. 2020. texSketch: Active Diagramming through Pen-and-Ink Annotations. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3313831.3376155>
 - [40] Ivan E. Sutherland. 1963. Sketchpad: a man-machine graphical communication system. In *Proceedings of the May 21–23, 1963, spring joint computer conference (AFIPS '63 (Spring))*. Association for Computing Machinery, New York, NY, USA, 329–346. <https://doi.org/10.1145/1461551.1461591>
 - [41] William Robert Sutherland. 1966. *The on-line graphical specification of computer procedures*. Thesis. Massachusetts Institute of Technology. <https://dspace.mit.edu/handle/1721.1/13474> Accepted: 2005-09-21T22:40:43Z.
 - [42] Ryo Suzuki, Rubaiat Habib Kazi, Li-yi Wei, Stephen DiVerdi, Wilmot Li, and Daniel Leithinger. 2020. RealitySketch: Embedding Responsive Graphics and Visualizations in AR through Dynamic Sketching. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology (UIST '20)*. Association for Computing Machinery, New York, NY, USA, 166–181. <https://doi.org/10.1145/3379337.3415892>
 - [43] Phil Turner. 2016. A Make-Believe Narrative for HCI. In *Digital Make-Believe*, Phil Turner and J. Tuomas Harviainen (Eds.). Springer International Publishing, Cham, 11–26. https://doi.org/10.1007/978-3-319-29553-4_2
 - [44] Barbara Tversky. 2011. Visualizing Thought. *Topics in Cognitive Science* 3, 3 (2011), 499–535. <https://doi.org/10.1111/j.1756-8765.2010.01113.x> eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1756-8765.2010.01113.x>
 - [45] Bret Victor. 2014. Humane representation of thought: a trail map for the 21st century. In *Proceedings of the 27th annual ACM symposium on User interface software and technology (UIST '14)*. Association for Computing Machinery, New York, NY, USA, 699. <https://doi.org/10.1145/2642918.2642920>
 - [46] Terry Winograd. 1972. Understanding natural language. *Cognitive Psychology* 3, 1 (Jan. 1972), 1–191. [https://doi.org/10.1016/0010-0285\(72\)90002-3](https://doi.org/10.1016/0010-0285(72)90002-3)
 - [47] Haijun Xia. 2020. Crosspower: Bridging Graphics and Linguistics. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology (UIST '20)*. Association for Computing Machinery, New York, NY, USA, 722–734. <https://doi.org/10.1145/3379337.3415845>
 - [48] Haijun Xia, Tony Wang, Aditya Gunturu, Peiling Jiang, William Duan, and Xiaoshuo Yao. 2023. CrossTalk: Intelligent Substrates for Language-Oriented Interaction in Video-Based Communication and Collaboration. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*. Association for Computing Machinery, New York, NY, USA, 1–16. <https://doi.org/10.1145/3586183.3606773>